Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Структури даних та алгоритми
Лабораторна робота №3

«Стеки та черги»

Виконав:
студент групи ІО-61
Астаф'єв Олександр Владиславович
Номер у списку групи: 2
Перевірив:
Русінов Володимир Володимирович

Київ-2026

Мета лабораторної роботи:

Засвоєння теоретичного матеріалу та набуття практичних навичок реалізації й застосування абстрактних структур даних стек (LIFO — «останнім прийшов, першим вийшов») та черга (FIFO — «першим прийшов, першим вийшов»), а також порівняння двох способів їхньої реалізації — на основі масиву фіксованого розміру та на основі однозв'язного списку.

Постанова задачі:

Кожен варіант завдання визначає прикладну задачу (наприклад, перевірку збалансованості дужок, обчислення постфіксного виразу тощо) та структуру(и) даних, потрібну для її розв'язання (стек, чергу, або обидві).

Для розв'язання задачі написати дві програми, що реалізують ту саму структуру даних двома способами:

на основі одновимірного масиву фіксованого розміру (з окремими змінними/індексами для вершини стека або голови й хвоста черги);

на основі однозв'язного списку (динамічне виділення пам'яті: вузол зі значенням і вказівником на наступний вузол).

Прикладна логіка (сама задача варіанта) в обох програмах має бути однаковою — відрізнятися має лише внутрішня реалізація стека/черги.

Обидві реалізації мають надавати єдиний узгоджений набір функцій, наприклад для стека: push, pop, peek, is_empty, is_full; для черги: enqueue, dequeue, peek_front, is_empty, is_full. Прикладна логіка (main і функції задачі) має викликати ці функції однаково незалежно від того, яка реалізація підключена — тобто дві програми відрізняються лише файлом/блоком, де реалізовані самі функції структури даних.

Розмір масиву для масивної реалізації взяти самостійно, достатнім для тестових прикладів (наприклад, 50–100 елементів), і оформити як іменовану константу.

Масивна реалізація черги обов'язково має бути кільцевою (циклічною): після заповнення останньої комірки масиву черга повинна продовжувати використовувати комірки, звільнені операціями dequeue, починаючи знову з початку масиву, а не вважати масив вичерпаним.

2	Стеком перевірити правильність вкладення дужок одного виду () у рядку та додатково визначити **максимальну глибину вкладення** (найбільшу кількість одночасно відкритих дужок).

Текст програми 1:

```
#include <stdio.h>

#include <string.h>
```

```
#define STACK_SIZE 50

typedef struct {
    char data[STACK_SIZE];
    int top;
} Stack;

void init(Stack *s) { s->top = -1; }

int is_empty(Stack *s) { return s->top == -1; }

int is_full(Stack *s) { return s->top == STACK_SIZE - 1; }

int push(Stack *s, char x)
{
    if (is_full(s))
        return 0;
    s->top++;
    s->data[s->top] = x;
    return 1;
}

int pop(Stack *s, char *x)
{
    if (is_empty(s))
        return 0;
    *x = s->data[s->top];
    s->top--;
    return 1;
}
```

```
int peek(Stack *s, char *x)
{
    if (is_empty(s))
        return 0;
    *x = s->data[s->top];
    return 1;
}

int main(void)
{
    Stack st;
    char s[256];
    char c;
    int i, depth = 0, max = 0, ok = 1;

    printf("Enter a string: ");
    if (fgets(s, sizeof(s), stdin) == NULL)
        return 0;
    s[strcspn(s, "\r\n")] = '\0';

    init(&st);
    for (i = 0; s[i] != '\0' && ok; i++)
    {
        if (s[i] == '(')
        {
            if (push(&st, '('))
            {
                depth++;
                if (depth > max)
                    max = depth;
            }
```

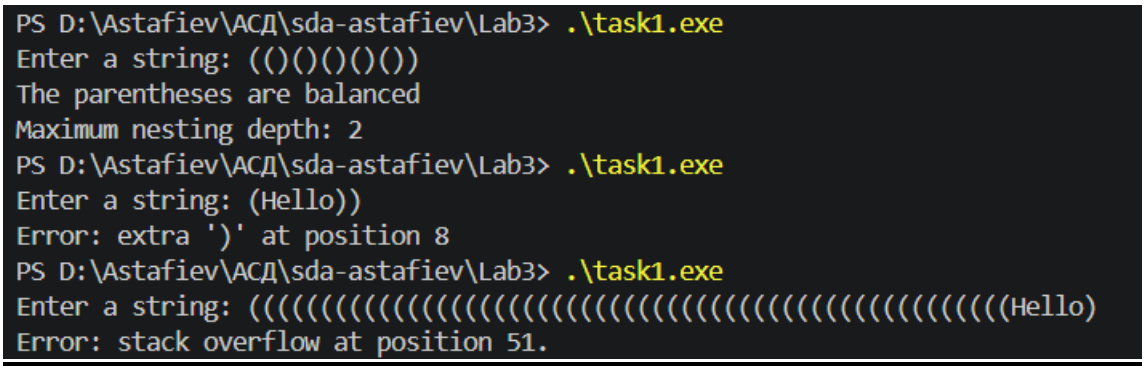
```
        else
        {
            printf("Error: stack overflow at position %d.\n", i + 1);
            ok = 0;
         }
    }
    else if (s[i] == ')')
    {
        if (pop(&st, &c))
            depth--;
        else
        {
            printf("Error: extra ')' at position %d\n", i + 1);
            ok = 0;
        }
    }
}

if (ok && !is_empty(&st))
{
    printf("Error: there is an unclosed '('.\n");
    ok = 0;
}
if (ok)
{
    printf("The parentheses are balanced\n");
    printf("Maximum nesting depth: %d\n", max);
}

while (!is_empty(&st))
    pop(&st, &c);
```

```
    return 0;
}
```

Результати програми 1:



Текст програми 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char data;
    struct Node *next;
} Node;

typedef struct {
    Node *top;
} Stack;

void init(Stack *s) { s->top = NULL; }

int is_empty(Stack *s) { return s->top == NULL; }

int is_full(Stack *s)
{
    (void)s;
```

```
    return 0;
}

int push(Stack *s, char x)
{
    Node *n = (Node *)malloc(sizeof(Node));
    if (n == NULL)
        return 0;
    n->data = x;
    n->next = s->top;
    s->top = n;
    return 1;
}

int pop(Stack *s, char *x)
{
    Node *n;
    if (is_empty(s))
        return 0;
    n = s->top;
    *x = n->data;
    s->top = n->next;
    free(n);
    return 1;
}

int peek(Stack *s, char *x)
{
    if (is_empty(s))
        return 0;
    *x = s->top->data;
```

```
    return 1;
}

int main(void)
{
    Stack st;
    char s[256];
    char c;
    int i, depth = 0, max = 0, ok = 1;

    printf("Enter a string: ");
    if (fgets(s, sizeof(s), stdin) == NULL)
        return 0;
    s[strcspn(s, "\r\n")] = '\0';

    init(&st);
    for (i = 0; s[i] != '\0' && ok; i++)
    {
        if (s[i] == '(')
        {
            if (push(&st, '('))
            {
                depth++;
                if (depth > max)
                    max = depth;
            }
            else
            {
                printf("Error: failed to allocate memory for a new node (position %d).\n", i
+ 1);
                ok = 0;
            }
```

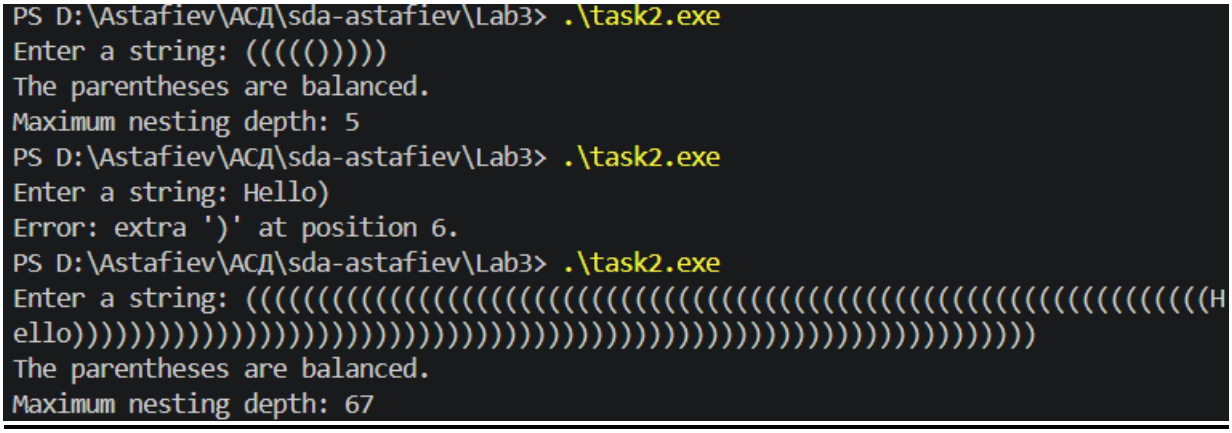
```
        }
        else if (s[i] == ')')
        {
            if (pop(&st, &c))
                depth--;
            else
            {
                printf("Error: extra ')' at position %d.\n", i + 1);
                ok = 0;
            }
        }
    }

    if (ok && !is_empty(&st))
    {
        printf("Error: there is an unclosed '('.\n");
        ok = 0;
    }
    if (ok)
    {
        printf("The parentheses are balanced.\n");
        printf("Maximum nesting depth: %d\n", max);
    }

    while (!is_empty(&st))
        pop(&st, &c);

    return 0;
}
```

Результати програми 2:


PS D:\Astafiev\АСД\sda-astafiev\Lab3> .\task2.exe
Enter a string: (((((())))))
The parentheses are balanced.
Maximum nesting depth: 5
PS D:\Astafiev\АСД\sda-astafiev\Lab3> .\task2.exe
Enter a string: Hello)
Error: extra ')' at position 6.
PS D:\Astafiev\АСД\sda-astafiev\Lab3> .\task2.exe
Enter a string: ((((((